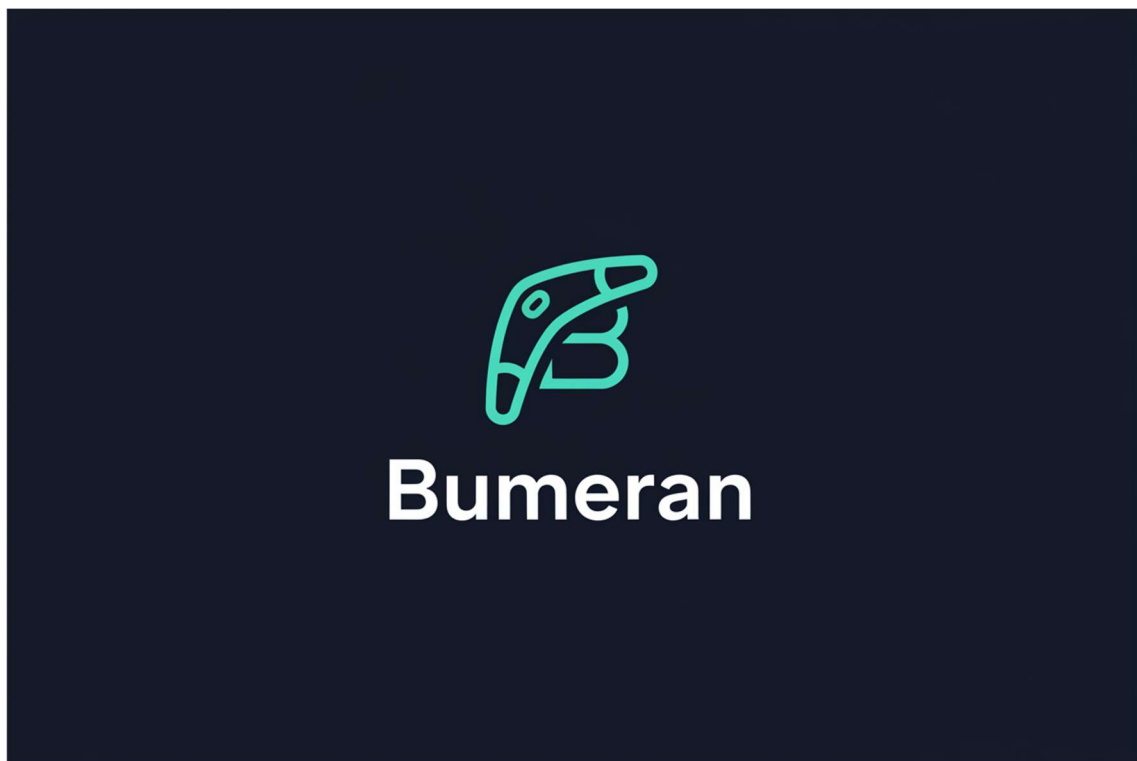


DOCUMENTACIÓN TÉCNICA PROYECTO INTERMODULAR:

BUMERAN



lucia Pastor Morales

A) DESCRIPCIÓN DEL PROYECTO

Bumeran es una herramienta de gestión de inventario y préstamos diseñada para **círculos de confianza** (familia, amigos o vecinos cercanos). Con ella podemos disfrutar compartiendo de aquellos objetos de uso ocasional que se suelen prestar, evitando que no vuelvan a llegar a nuestras manos. Con ella logramos algunos objetivos como:

- **Control de pertenencias:** Saber en todo momento quién tiene un objeto tuyo, evitando que las cosas "se pierdan" al prestarlas.
- **Consumo responsable:** Fomentar que los grupos de amigos o familiares compartan objetos de uso ocasional (herramientas, material de acampada) en lugar de que cada uno compre el suyo.
- **Seguridad y confianza:** Limitar el intercambio a personas conocidas para asegurar que los objetos se cuiden y se devuelvan en buen estado.

La aplicación funciona como un registro privado de objetos. Un usuario sube aquellos artículos que suele prestar y no quiere perder. Cuando este realiza un préstamo se asocia a la persona a la que le ha sido prestado: si es de la aplicación mediante su usuario así el también podrá ver el estado del préstamo, y si no dispone de esta herramienta, podrá asociarse al nombre que el usuario desee. Bumeran nos permitirá ver el estado del préstamo, cuanto tiempo lleva tu objeto sin estar junto a ti y el nombre de la persona que lo esta usando.

Es, en esencia, un panel de control para que el propietario nunca pierda el rastro de sus cosas.

Para empezar a crear Bumeran, primero reuní todos los requisitos que quería que tuviese y , a partir de ahí, fui viendo como se podían implementar y cuales eran viables con mis conocimientos. Por lo que los requisitos funcionales que concluí fueron los siguientes:

- La aplicación debe permitir el registro de usuarios con nombre, email y contraseña.
- El sistema debe autenticar a los usuarios mediante formulario de login.
- Los usuarios autenticados deben poder crear, listar y eliminar sus propios artículos.
- Los usuarios deben poder prestar sus artículos a otros usuarios registrados o a personas externas introduciendo su nombre manualmente para así ampliar más la funcionalidad de la aplicación.
- El sistema debe llevar un registro de la fecha de inicio del préstamo y permitir marcar la devolución, registrando la fecha de fin.
- Los préstamos activos deben mostrar los días transcurridos en tiempo real.
- El dashboard debe diferenciar visualmente los artículos prestados de los disponibles.
- Los artículos prestados no deben poder eliminarse hasta que sean devueltos.

A nivel técnico fue más fácil marcar los requisitos tras las indicaciones que se marcan a seguir en el proyecto, por lo que obtuvimos estos:

- Backend desarrollado con Java 21 y Spring Boot 4.1.
- Frontend en HTML/CSS cono Thymeleaf.
- Base de datos MySQL 8 con al menos 3 tablas relacionadas (usuarios, artículos, préstamos).
- Contraseñas almacenadas con BCrypt .
- Autenticación mediante Spring Security con formulario web.
- Despliegue en AWS Academy EC2 con Ubuntu.
- Contenedores Docker gestionados con Docker Compose (3 contenedores: nginx, app, mysql).
- Sistema de backups completo (semanal full + diaria incremental) programada con cron.
- Repositorio Git público con control de versiones.

Finalmente las funcionalidades que surgieron fueron las siguientes:

- Registro de usuarios: formulario con validación de email único y contraseña mínima de 6 caracteres cifrada con BCrypt.
- Inicio de sesión: mediante formulario web.
- Dashboard personal: cada usuario ve solo sus artículos con indicador un de estado (DISPONIBLE / PRESTADO).
- Gestión de artículos: crear, listar y eliminar artículos propios (con protección para no eliminar artículos prestados).
- Creación de préstamos: seleccionar artículo propio disponible y elegir entre usuario registrado o introducir nombre manualmente.
- Registro de devoluciones: marcar un artículo como devuelto, actualizando su estado a DISPONIBLE y registrando la fecha de fin.
- Cálculo automático de días prestados: muestra los días transcurridos desde el inicio del préstamo.
- Sección "Items que me han prestado": para poder visualizar los artículos que otros usuarios te han prestado a ti.
- Interfaz en inglés: login, dashboard, loans y creación de artículos/préstamos están en inglés (register en castellano para facilitar el registro).
- Backups automáticos: copia completa cada domingo y copia incremental diaria, con retención de 30 y 15 días respectivamente.

Bumeran no está orientada a empresas ni a uso comercial, sino que está dirigida a particulares que desean llevar un control organizado de los objetos que prestan a amigos, familiares o conocidos.

El perfil de usuario típico es cualquier persona que preste objetos con frecuencia (herramientas, libros, dispositivos electrónicos, prendas de vestir, etc.) y necesite recordar quién tiene qué, desde cuándo y cuánto tiempo lleva prestado. Como podría darse en grupos de amigos, familias o comunidades de vecinos pequeñas. Además, los usuarios pueden ser tanto la persona que presta (propietario del artículo) como la que recibe el préstamo (receptor), permitiendo que ambos tengan visibilidad del estado del préstamo.

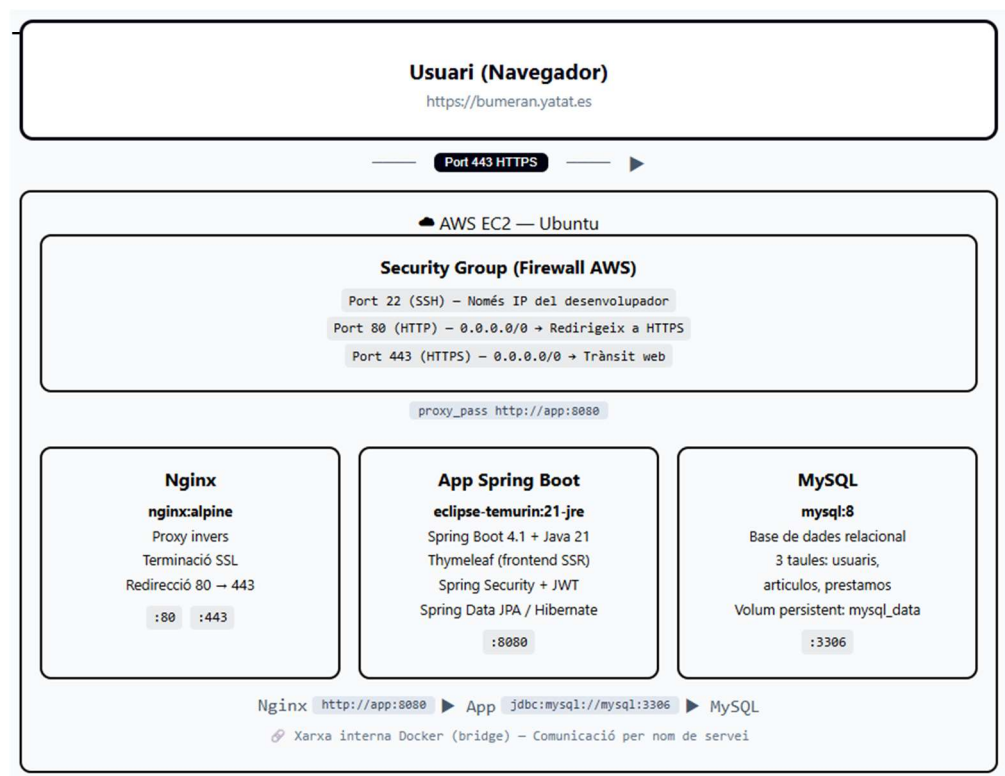
Como ejemplos de algunos casos de usos de esta aplicación podríamos distinguir estos tres ejemplos:

Crear un préstamo: donde el usuario selecciona entre sus objetos aquel que va a prestar y a continuación a quién lo va hacer, si se trata de un usuario de bumeran buscará su nombre y si es una persona externa pondrá como quiere dirigirse a ella. Una vez creado el préstamo aparecerá con un estado 'PRESTADO' junto a la fecha de inicio y a quien ha sido prestado dicho artículo.

Crear artículo: El usuario registrado accionará sobre 'ADD ITEM' y a continuación se añadirá el nombre del artículo junto a una descripción. Una vez guardado se le asignará el estado 'DISPONIBLE' y aparecerá en el dashboard.

Eliminar artículo: Se pulsará sobre el icono de eliminar que se encuentra en el dashboard junto al artículo y este se eliminará de la Base de Datos

B) STACK TECNOLÓGICO



He dividido el proyecto en dos bloques:

El frontend donde he trabajado con Thymeleaf (HTML + CSS) y el backend, desarrollado en SpringBoot con Java como lenguaje principal.

Ambas partes se comunican endpoints JSON en /api/ para ser consumidos por clientes externos (apps móviles, postman, etc.), y para las vistas backend genera el HTML completo y lo envía al navegador.

Específicamente y explicándolo de manera más amplia, he implementado las siguientes tecnologías:

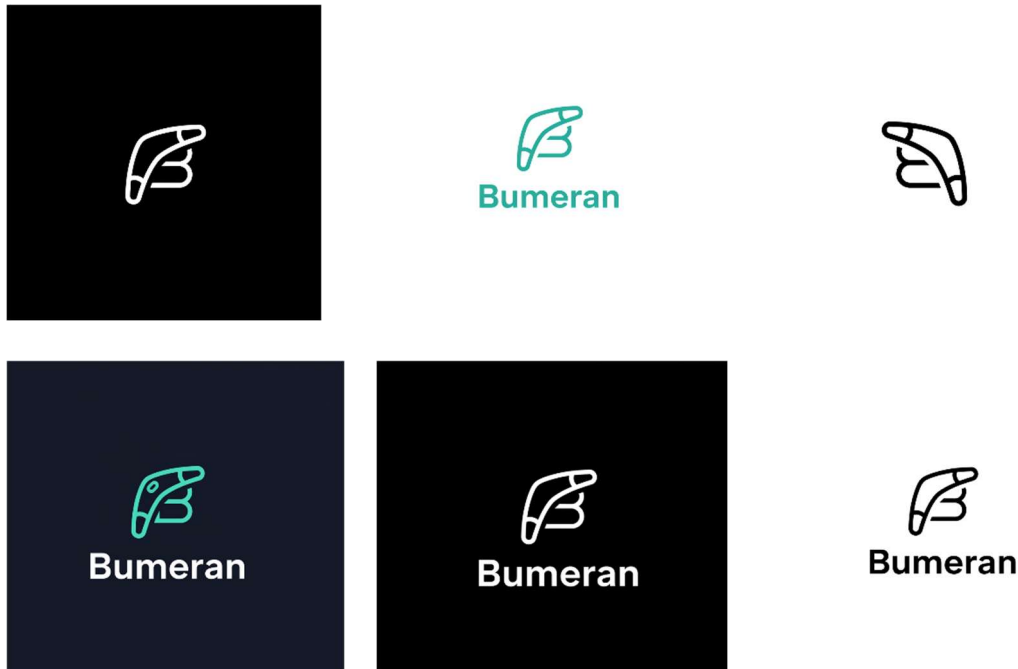
- Java 21: como lenguaje de programación en el backend
- SpringBoot 4.1: Framework principal
- Spring Security 6.x: Spring Security
- Spring Data JPA 3.x: Acceso a base de datos (ORM)
- Hibernate 6.x: Mapeo objeto-relacional
- Thymeleaf 3.x: Frontend
- MYSQL 8.0: Base de Datos
- JWT 0.11: Tokens de autenticación para API REST
- BCrypt: Cifrado de contraseñas
- Lombok 1.18: Generación automática de getters/setters
- Maven 3.9; Gestión de dependencias y build
- Docker 24+: Contenedores
- Docker Compose 2.x: Orquestación de contenedores
- Nginx 1.25+: Servidor web / proxy inverso
- GPG 2.4: Cifrado de backups
- Cerbot: Certificado SSL gratuito
- AWS EC2: Servidor cloud (Ubuntu)
- GitHub: Control de versiones

Los puertos en uso para nuestro proyecto son:

- 22/SSH: Para poder conectarse de manera remota al EC2
- 80/Nginx: Redirección inmediata a HTTPS.
- 443: como punto de entrada a la aplicación.
- 8080: para poder acceder a Springboot desde nginx.
- 3306: Acceder a la base de datos.

C) IDENTIDAD VISUAL Y DISEÑO

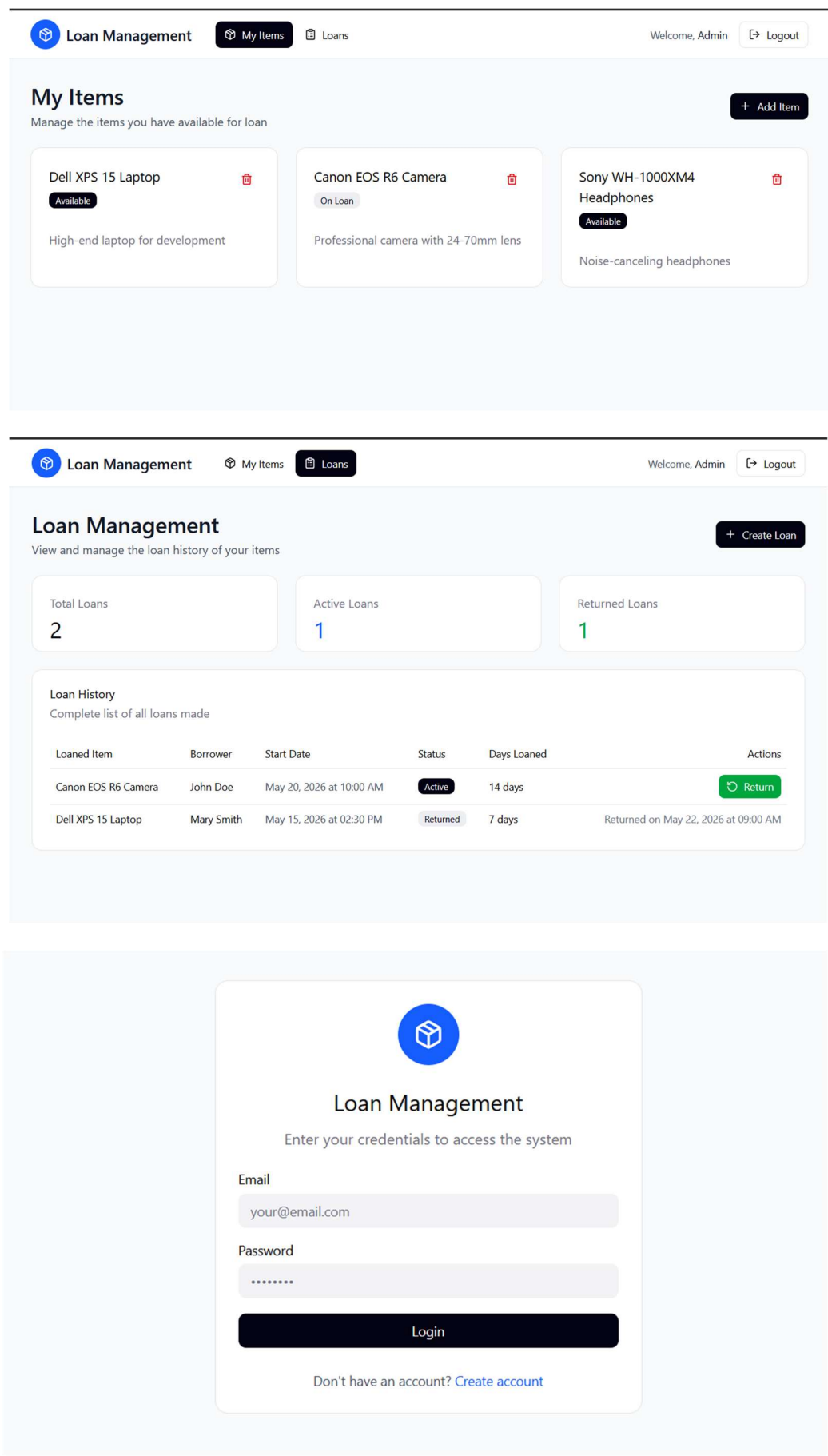
Para bumeran contamos con dos versiones del logo: una versión completa y el icono derivado del logo en varias versiones:



Para la paleta de colores de Bumeran he optado por colores neutros y claros para conseguir un estilo minimalista y profesional, priorizando la legibilidad y la claridad visual por ellos los colores empleados son: Blanco, gris y negro para la pagina (registro, creación de artículos, proyectos, etc), rojo para los mensajes de erros, verde para marcar como finalizado el préstamo, amarillo para indicar si un objeto está prestado y azul para los fondos de las tarjetas de los prestamos recibidos.

En cuanto a la tipografía, al ser una página funcional que no busca el diseño sino la practicidad hemos optado por font-family, y sus derivados para cada sistema operativo, así optimizamos el rendimiento que nos supondría cargar una fuente externa.

Como un primer prototipo y fuente de inspiración, creé un primer diseño en Figma:

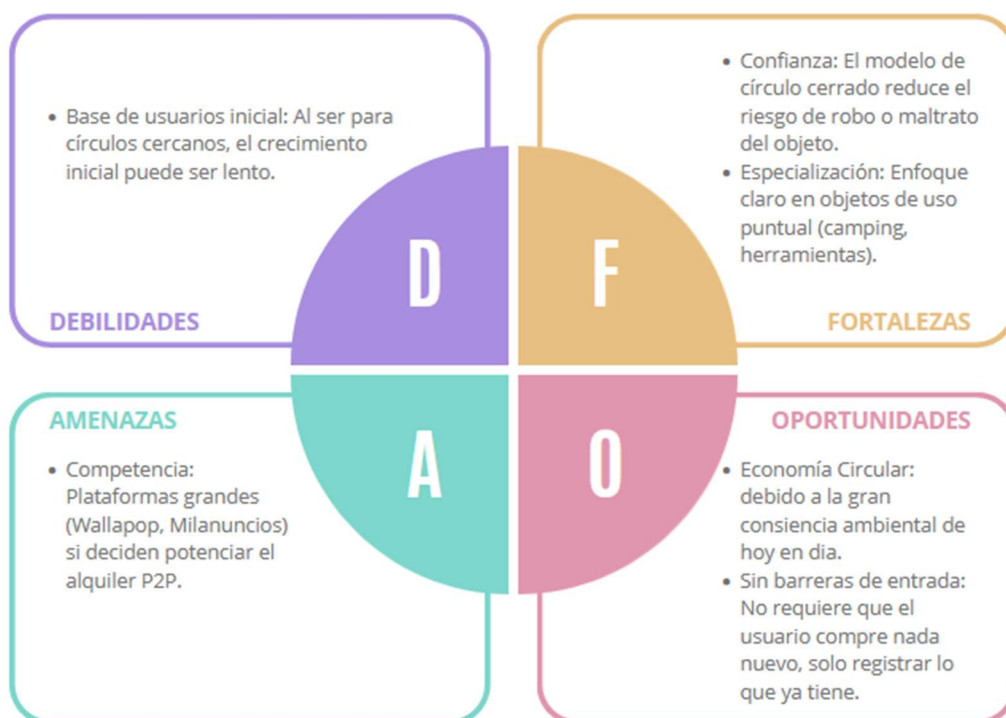


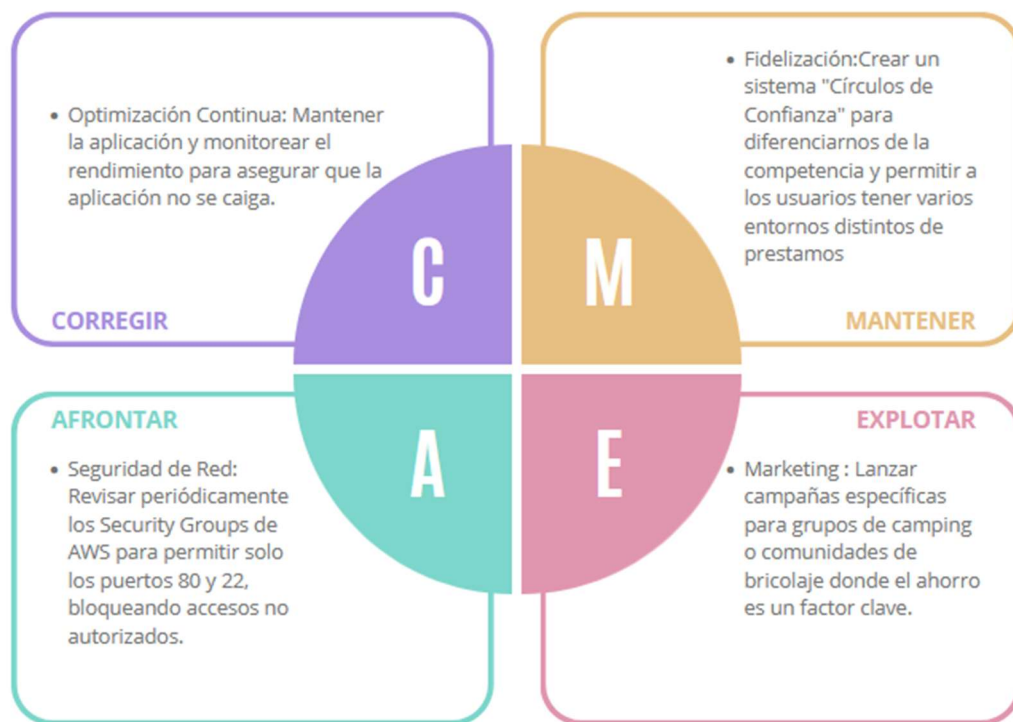
D) PLAN DE NEGOCIO

Para empezar con la viabilidad económica de Bumeran partimos con la ventaja de que la aplicación tiene unos costes de inicio prácticamente nulos , ya que toda la infraestructura se ha montado usando herramientas de código abierto como Docker y Spring Boot. Además al estar subida en una instancia de AWS podemos aprovechar los créditos de estudiante o el nivel gratuito por lo que no tenemos que invertir dinero real en servidores potentes desde el primer día

En cuanto a la forma de ganar dinero con la plataforma, aunque nos centramos en préstamos dentro de un círculo cercano de amigos y familiares, el proyecto es totalmente viable si implementamos un modelo freemium donde la app sea gratis para gestionar unos pocos objetos pero cobremos una pequeña suscripción si un grupo quiere organizar un inventario muy grande.

Otra opción sería meter publicidad muy localizada de tiendas de barrio o enlaces de afiliados para comprar repuestos como cartuchos de gas para el camping o brocas para los taladros aprovechando que la app ya sabe qué objetos se están prestando más en ese momento.





E) INFRAESTRUCTURA Y DESPLIEGE

Para este proyecto, el dominio que utilizamos es `bumeran.yatat.es` que se asocia con la IP elástica de nuestro EC2.

En cuanto al despliegue en AWS, lo primero fue crear la máquina virtual desde la consola de AWS. Elegimos:

- Sistema operativo: Ubuntu 22.04 LTS
- Tipo de instancia: t2.medium
- Par de claves: descargamos un fichero `.pem` para conectarnos por SSH

Para conectarme al SSH Desde la terminal (PowerShell en Windows), nos conectamos: `ssh -i "C:\Users\lucia\Desktop\clave.pem" ubuntu@52.22.119.83`
Y ya estamos dentro de la máquina virtual, como si fuera un Ubuntu normal pero en la nube.

Luego instalamos Docker, ya que Ubuntu no lo trae por defecto. Una vez instalado, subimos nuestro código para poder trabajar con el realizando un clone a nuestro repositorio de github.

Una vez el código estaba en el EC2, ya podíamos ejecutar Docker Compose el cual construye las imágenes (la de la app con Maven), baja MySQL y Nginx de los repositorios oficiales, y arranca todo.

Para crearlo, no fue a la primera ya que tuve complicaciones con los permisos de Docker, MySQL no arrancaba a tiempo y Thymeleaf no encontraba las vistas porque las rutas eran relativas.

Una vez se solucionaron los errores instalamos el certificado digital para tener HTTPS. Los pasos para conseguirlo fueron los siguientes:

- Abrir el port 443 en el Security Group, pero como AWS tiene un firewall delante del EC2 que se llama Security Group, solo deja pasar el puerto 22 (SSH) así que añadí una regla para el puerto 443 (HTTPS).
- Instalar Certbot en el EC2.
- Detener Nginx temporalmente ya que Certbot necesita el puerto 80 para verificar que controlamos el dominio
- Obtener el certificado
- Copiar los certificados a una ubicación accesible
- Configurar Nginx con HTTPS. Para ello creamos un fichero nginx-ssl.conf que dice a Nginx que escuche en el puerto 443 y use los certificados.
- Montar los certificados en el contenedor. Actualizamos el docker-compose.yml para montar la carpeta de los certificados dentro del contenedor.
- Renovación automática: como los certificados de Let's Encrypt caducan cada 90 días programamos una tarea con cron.

Para llevar el control de versiones y poder trabajar desde cualquier sitio, utilizamos Git y GitHub.

Desarrollábamos localmente en Windows (con IntelliJ, VS Code, etc.), cuando teníamos una versión estable, hacíamos *commit* y *push* al repositorio GitHub, en el EC2, hacíamos *pull* para actualizar el código, reconstruíamos la imagen de la app con `docker compose build app` y arrancábamos de nuevo con `docker compose up -d`.

En el caso de nuestro repositorio se pueden distinguir tres bloques claros:

- Backend/ contiene todo el código Java. Dentro de src/main/java, las clases están separadas por carpetas: Controllers, Servicios, Repositorías, Entidades, Config, dto. Cada una hace algo concreto.
- frontend-thymeleaf/ donde se encuentra todo lo que ve el usuario en el navegador: las páginas HTML, el CSS y las imágenes.
- nginx/ contiene la configuración del servidor web que hace de puente entre Internet y la app.
- scripts/ contiene las herramientas para realizar backups automáticos de la base de datos.

F) POLÍTICA DE COPIAS DE SEGURIDAD

Las copias de seguridad sirven para guardar un registro de archivos como la bd por si ocurriese algún error o se perdiesen datos poder tener una versión actualizada o recuperarlos. Ya que si el servidor se rompe, alguien borra la base de datos por error o MySQL se corrompe, sin copia se perdería todo: usuarios, artículos, préstamos y días contabilizados. Para evitar este desastre, diseñamos una política de backups basada en dos tipos de copias:

- Una copia completa, es decir, guarda todo durante treinta días y esta se realiza cada domingo a las 2 de la mañana.
- Copia incremental que solo guarda los cambios ocurridos desde la última copia utilizando los binary logs de MySQL, que registran cada operación que modifica datos (INSERT, UPDATE, DELETE).

Hemos creado cuatro scripts situados en la carpeta scripts/ del repositorio para gestionar los backups:

- Script 1: backup-full.sh: Realiza la copia completa. Crea el directorio /backups/full/ (permisos 750), genera el dump con la fecha en el nombre mediante mysqldump --single-transaction (para no bloquear las tablas), cifra el archivo con GPG (AES256) usando la contraseña BumeranBackup2026 y elimina las copias de más de 30 días.
- Script 2: backup-incremental.sh: Realiza las copias diarias. Comprueba los binary logs, ejecuta un FLUSH LOGS para cerrar el log actual y lo copia al directorio de backups, guardando también la posición actual para restauraciones *point-in-time*.

- Script 3: setup-cron-backups.sh: Programa las tareas automáticas en el cron del sistema para que los scripts se ejecuten solos en los horarios establecidos.

Y por último, para la recuperación de datos utilizamos el Script 4: restore.sh, que sigue estos pasos:

1. Recibe el archivo de backup (.sql o .sql.gpg).
2. Descifra automáticamente el archivo si es necesario con la contraseña del proyecto.
3. Solicita la confirmación escrita "CONFIRM" para evitar errores.
4. Borra la base de datos actual (DROP DATABASE bumeran), la recrea e importa el dump.
5. Reinicia la aplicación (docker restart bumeran-app) para refrescar la conexión.